

# Design Verification Report

## RISC-V Single-Cycle CPU

### *Top-Level Integration Testbench*

Design Author: Elliot Staresinic

DUT: sc\_cpu\_top\_level | Stimulus: program.mem (30 instructions)

Simulator: Icarus Verilog (iverilog -g2012)

Date: 2026-05-31 | Status: PASS

**VERIFICATION RESULT: PASS (0 errors)**

# Contents

---

|                                                  |    |
|--------------------------------------------------|----|
| Contents .....                                   | 2  |
| 1. Executive Summary .....                       | 3  |
| 2. Design Under Test .....                       | 3  |
| 2.1 Supported Instruction Subset.....            | 3  |
| 2.2 Module Hierarchy .....                       | 3  |
| 3. Verification Methodology and Testbench.....   | 4  |
| 3.1 Key Parameters.....                          | 4  |
| 3.2 Lockstep Golden Model .....                  | 4  |
| 3.3 Final-State Oracle.....                      | 4  |
| 3.4 Supported Instruction Subset.....            | 5  |
| 3.5 Halt Condition and Watchdog .....            | 5  |
| 3.6 Waveform Dump.....                           | 5  |
| 3.7 Per-Instruction Trace and Disassembler ..... | 5  |
| 3.8 Hierarchical References .....                | 5  |
| 3.9 Verification Task Reference.....             | 6  |
| 4. Program Under Test.....                       | 6  |
| 4.1 Execution Trace.....                         | 6  |
| 5. Verification Results .....                    | 7  |
| 5.1 Lockstep Check.....                          | 7  |
| 5.2 Final-State Oracle.....                      | 7  |
| 5.3 Final Register File.....                     | 8  |
| 5.4 Final Data Memory (non-zero words).....      | 8  |
| 6. Functional Coverage .....                     | 9  |
| 6.1 Coverage Matrix .....                        | 9  |
| 6.2 Dead-Code Lines.....                         | 9  |
| 7. Issues and Recommendations .....              | 10 |
| 7.1 Open Issues .....                            | 10 |
| 7.2 Recommendations.....                         | 10 |
| 8. Conclusion .....                              | 10 |
| Appendix A. Simulation Waveform                  |    |

## 1. Executive Summary

This report documents the functional verification of a single-cycle RISC-V CPU at the top level (module `sc_cpu_top_level`). The design under test (DUT) was exercised with a 30-instruction program and checked against two independent reference oracles. All checks passed: the DUT register file, data memory, and program counter matched the golden reference at every committed instruction, and the hand-derived end-of-program state matched as well.

Result: **PASS, 0 errors**. 28 instructions executed before clean halt; the design terminated without watchdog intervention.

This program is a strong functional test of the supported instruction subset: every instruction type is exercised, and critically, all three conditional branches (`beq`, `bne`, `blt`) are tested in both the taken and not-taken directions. A small number of coverage gaps remain and are listed in Section 6.

## 2. Design Under Test

The DUT is a non-pipelined, single-cycle RV32I-subset processor using a Harvard architecture (separate instruction and data memories). Each instruction completes in one clock cycle. The program counter is reset asynchronously and advances on the rising clock edge.

### 2.1 Supported Instruction Subset

| Category     | Instructions           | Notes                                |
|--------------|------------------------|--------------------------------------|
| R-type       | add, sub, and, or, slt | funct7 bit 5 selects sub vs. add     |
| I-type       | addi, andi, ori, slti  | Immediate arithmetic; slti decodable |
| Load / Store | lw, sw                 | Word-aligned, computed base + offset |
| Branch       | beq, bne, blt          | One-hot branch control; PC-relative  |

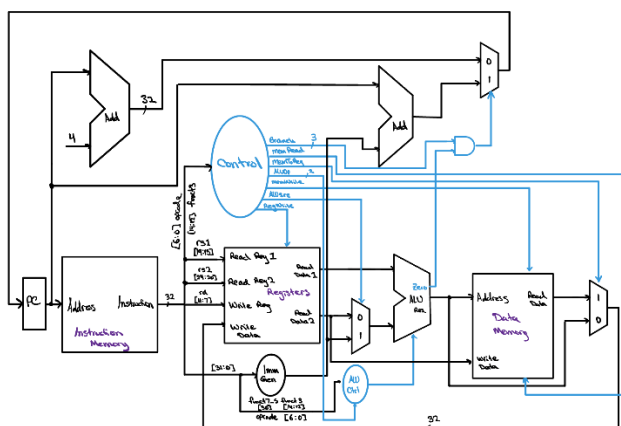


Figure 1: System Architecture

## 2.2 Module Hierarchy

|                           |                                                           |
|---------------------------|-----------------------------------------------------------|
| sc_cpu_top_level          |                                                           |
| ├─ sc_cpu_control         | Main control unit (combinational, opcode-decoded)         |
| ├─ alu_control            | ALU control unit (funct3/funct7 decode)                   |
| ├─ sc_cpu_datapath        |                                                           |
| │   └─ instruct_mem       | Read-only instruction memory (loaded from program.mem)    |
| │   └─ imm_gen            | Immediate generator (I, S, SB, U, UJ formats)             |
| │   └─ reg_file           | 32x32-bit register file (2 read ports, 1 write port)      |
| │   └─ mux_2x1            | ALU source mux; write-back mux; branch/increment mux (x3) |
| │   └─ alu_full           | 32-bit ALU (ripple-carry, parameterized)                  |
| │       └─ alu_slice      | 1-bit ALU slice (bits [30:0])                             |
| │           └─ mux_2x1    | Output select (ALU result vs. pass-through)               |
| │           └─ mux_4x1    | Operation select (add/sub, and, or, slt)                  |
| │           └─ full_adder | 1-bit full adder                                          |
| │       └─ alu_msb        | MSB ALU slice (overflow detection, setless)               |
| │           └─ mux_2x1    | Output select (ALU result vs. pass-through)               |
| │           └─ mux_4x1    | Operation select (add/sub, and, or, slt)                  |
| │           └─ full_adder | 1-bit full adder (MSB, feeds overflow logic)              |
| ├─ data_mem               | Clocked data memory (word-aligned, read/write enable)     |
| └─ ripple_carry_adder     | PC + 4 adder; PC + immediate adder (x2)                   |

## 3. Verification Methodology and Testbench

The testbench (`tb_sc_cpu_top_level`) is a self-checking, top-level integration testbench targeting Icarus Verilog (`iverilog -g2012`). Verification uses two independent oracles so that a bug in either the design or the checking code is unlikely to escape undetected.

### 3.1 Key Parameters

| Parameter  | Value / Purpose                                                                    |
|------------|------------------------------------------------------------------------------------|
| CLK_PERIOD | 10 ns — drives the always block that toggles clk                                   |
| NUM_INSTR  | 30 — halt boundary; simulation stops when <code>ref_pc</code> word index $\geq 30$ |
| DMEM_WORDS | 256 — number of 32-bit data-memory words checked each step                         |
| IMEM_WORDS | 256 — reference instruction-memory size (mirrored from <code>program.mem</code> )  |
| MAX_STEPS  | 1000 — runaway watchdog limit                                                      |

### 3.2 Lockstep Golden Model

A behavioral RV32I-subset simulator (task `ref_step`) decodes and executes one RV32I instruction per call, maintaining its own `ref_regs[0:31]`, `ref_dmem[0:255]`, and `ref_pc`. It runs one instruction ahead of each DUT clock edge, encoding the correct architectural semantics. After every posedge `clk` the task `check_state` compares the DUT PC (`dut.datapath.pc`), all 32 register-file entries (`dut.datapath.registers.RF[k]`), and all 256 data-memory words (`dut.datapath.data_memory.memory[k]`) against the reference. Any mismatch is logged immediately with step number and field name.

### 3.3 Final-State Oracle

The tasks `load_expected` and `check_final` encode a hand-derived, independently computed table of the expected end-of-program register and memory state, checked once after halt. Because it

is produced by a completely different method from the lockstep model (manual trace), agreement among DUT, lockstep model, and this table gives high confidence that a latent bug in the lockstep model itself has not masked a DUT defect.

### 3.4 Supported Instruction Subset

The golden model (task `ref_step`) decodes and executes the following RV32I instructions:

| Opcode group            | Instructions           | Notes                                         |
|-------------------------|------------------------|-----------------------------------------------|
| OPC_RTYPE<br>(0110011)  | add, sub, and, or, slt | funct7[5] selects sub; signed compare for slt |
| OPC_ITYPE<br>(0010011)  | addi, andi, ori, slti  | 12-bit sign-extended immediate                |
| OPC_LOAD<br>(0000011)   | lw                     | Byte address = $rs1 + imm_i$ ; word-aligned   |
| OPC_STORE<br>(0100011)  | sw                     | Byte address = $rs1 + imm_s$ ; word-aligned   |
| OPC_BRANCH<br>(1100011) | beq, bne, blt          | PC-relative; signed compare for blt           |

### 3.5 Halt Condition and Watchdog

The main execution loop terminates when `ref_pc[31:2] >= NUM_INSTR (30)`. Because the DUT uses the same program image and identical control flow, it reaches the same halt condition on the same clock edge. A global watchdog (initial block with absolute delay) triggers `$finish` after `CLK_PERIOD * (MAX_STEPS + 50)` nanoseconds if the loop has not already exited, preventing the simulator from hanging on a runaway design.

### 3.6 Waveform Dump

The testbench calls `$dumpfile("waveforms/dump.vcd")` and `$dumpvars(0, tb_sc_cpu_top_level)` in an initial block, producing a Value Change Dump covering the full hierarchy. The VCD is viewable in EPWave (EDA Playground) or any compatible waveform viewer such as GTKWave.

### 3.7 Per-Instruction Trace and Disassembler

After each step the task `trace_print` emits a formatted line containing the step counter, PC (hex), instruction word (hex), a pseudocode disassembly produced by the task `disasm`, and a plain-English description of the architectural effect (register write, store, branch taken/not taken, or no state change). The `disasm` task covers all opcodes in the supported subset and falls back to `.word 0xxxxxxxxx` for any unrecognized encoding.

### 3.8 Hierarchical References

The testbench accesses DUT internal state directly via hierarchical dot-notation paths, which must match the actual module and signal names in the RTL:

|                                                 |                                       |
|-------------------------------------------------|---------------------------------------|
| <code>dut.datapath.pc</code>                    | – program counter                     |
| <code>dut.datapath.registers.RF[k]</code>       | – register file array ( $k = 0..31$ ) |
| <code>dut.datapath.data_memory.memory[k]</code> | – data memory array ( $k = 0..255$ )  |

If the RTL uses different instance or signal names, these three paths must be updated accordingly for the testbench to compile and link.

### 3.9 Verification Task Reference

| Task             | Role                                                                                                               |
|------------------|--------------------------------------------------------------------------------------------------------------------|
| disasm           | Decode one 32-bit instruction word into pseudocode; covers all supported opcodes, falls back to .word for unknowns |
| ref_step         | Decode and execute one instruction in the behavioral model; update ref_regs, ref_dmem, ref_pc; set trace variables |
| check_state      | Compare DUT PC, all 32 registers, and all 256 data-memory words against the reference; print any mismatch          |
| trace_print      | Emit a human-readable per-instruction trace line: PC, hex encoding, pseudocode disassembly, effect                 |
| dump_final_state | Print the complete register file and all non-zero data-memory words at end of simulation                           |
| load_expected    | Populate the hand-derived final-state oracle (exp_regs, exp_dmem)                                                  |
| check_final      | Compare DUT final state against the oracle; report PASS or error count                                             |

## 4. Program Under Test

The 30-instruction program (`program.mem`) is loaded into both the DUT instruction memory and the reference model at simulation start via `$readmemh`. The execution trace below shows every committed instruction, the disassembled pseudocode, and the architectural effect. For detailed waveshape, see Appendix A.

### 4.1 Execution Trace

| Step | PC         | Encoding   | Pseudocode                   | Effect                                           |
|------|------------|------------|------------------------------|--------------------------------------------------|
| 1    | 0x00000000 | 0x00100093 | <code>addi x1, x0, 1</code>  | $x1 \leftarrow 0x00000001$ (1)                   |
| 2    | 0x00000004 | 0x00500113 | <code>addi x2, x0, 5</code>  | $x2 \leftarrow 0x00000005$ (5)                   |
| 3    | 0x00000008 | 0x002081B3 | <code>add x3, x1, x2</code>  | $x3 \leftarrow 0x00000006$ (6)                   |
| 4    | 0x0000000C | 0x40118233 | <code>sub x4, x3, x1</code>  | $x4 \leftarrow 0x00000005$ (5)                   |
| 5    | 0x00000010 | 0xFFE20293 | <code>addi x5, x4, -2</code> | $x5 \leftarrow 0x00000003$ (3)                   |
| 6    | 0x00000014 | 0x0042E333 | <code>or x6, x5, x4</code>   | $x6 \leftarrow 0x00000007$ (7)                   |
| 7    | 0x00000018 | 0x005273B3 | <code>and x7, x4, x5</code>  | $x7 \leftarrow 0x00000001$ (1)                   |
| 8    | 0x0000001C | 0x00522433 | <code>slt x8, x4, x5</code>  | $x8 \leftarrow 0x00000000$ (0) — $5 < 3$ false   |
| 9    | 0x00000020 | 0x0042A4B3 | <code>slt x9, x5, x4</code>  | $x9 \leftarrow 0x00000001$ (1) — $3 < 5$ true    |
| 10   | 0x00000024 | 0x00302223 | <code>sw x3, 4(x0)</code>    | $\text{dmem}[1] \leftarrow 0x00000006$           |
| 11   | 0x00000028 | 0x00800513 | <code>addi x10, x0, 8</code> | $x10 \leftarrow 0x00000008$ (8)                  |
| 12   | 0x0000002C | 0x00652223 | <code>sw x6, 4(x10)</code>   | $\text{dmem}[3] \leftarrow 0x00000007$           |
| 13   | 0x00000030 | 0x407105B3 | <code>sub x11, x2, x7</code> | $x11 \leftarrow 0x00000004$ (4)                  |
| 14   | 0x00000034 | 0xFFC52603 | <code>lw x12, -4(x10)</code> | $x12 \leftarrow \text{dmem}[1] = 0x00000006$ (6) |

| Step | PC         | Encoding   | Pseudocode       | Effect                                   |
|------|------------|------------|------------------|------------------------------------------|
| 15   | 0x00000038 | 0x00410663 | beq x2, x4, +12  | beq: x2=5 == x4=5 → TAKEN → 0x00000044   |
| —    | 0x0000003C | 0x003002B3 | add x5, x0, x3   | dead code — skipped by branch at step 15 |
| —    | 0x00000040 | 0x00F17693 | andi x13, x2, 15 | dead code — skipped by branch at step 15 |
| 16   | 0x00000044 | 0x00A02823 | sw x10, 16(x0)   | dmem[4] ← 0x00000008                     |
| 17   | 0x00000048 | 0x00F17713 | andi x14, x2, 15 | x14 ← 0x00000005 (5)                     |
| 18   | 0x0000004C | 0x00411A63 | bne x2, x4, +20  | bne: x2=5 != x4=5? NOT taken             |
| 19   | 0x00000050 | 0x00311463 | bne x2, x3, +8   | bne: x2=5 != x3=6 → TAKEN → 0x00000058   |
| —    | 0x00000054 | 0x407105B3 | sub x11, x2, x7  | dead code — skipped by branch at step 19 |
| 20   | 0x00000058 | 0x00610233 | add x4, x2, x6   | x4 ← 0x0000000C (12)                     |
| 21   | 0x0000005C | 0xFE1048E3 | blt x0, x1, -16  | blt: x0=0 < x1=1 → TAKEN → 0x0000004C    |
| 22   | 0x0000004C | 0x00411A63 | bne x2, x4, +20  | bne: x2=5 != x4=12 → TAKEN → 0x00000060  |
| 23   | 0x00000060 | 0x00326793 | ori x15, x4, 3   | x15 ← 0x0000000F (15)                    |
| 24   | 0x00000064 | 0x00208663 | beq x1, x2, +12  | beq: x1=1 == x2=5? NOT taken             |
| 25   | 0x00000068 | 0x00114463 | blt x2, x1, +8   | blt: x2=5 < x1=1? NOT taken              |
| 26   | 0x0000006C | 0x00902A23 | sw x9, 20(x0)    | dmem[5] ← 0x00000001                     |
| 27   | 0x00000070 | 0x00928013 | addi x0, x5, 9   | x0 write discarded (result = 0x0000000C) |
| 28   | 0x00000074 | 0x40238833 | sub x16, x7, x2  | x16 ← 0xFFFFFFF0 (-4)                    |

Instructions at PCs 0x3C, 0x40, and 0x54 are dead code: the branches at steps 15 and 19 cause execution to skip them entirely in this deterministic program.

## 5. Verification Results

### 5.1 Lockstep Check

The lockstep check fires after every clock edge and compares the DUT PC, all 32 registers, and all 256 data-memory words against the golden model. Zero mismatches were recorded across all 28 executed steps.

### 5.2 Final-State Oracle

The hand-derived oracle was compared against the DUT final state after halt. Zero mismatches were recorded. The complete expected and observed state is shown below.

### 5.3 Final Register File

| Register | Expected (hex) | Observed (hex) | Comment                                     |
|----------|----------------|----------------|---------------------------------------------|
| x0       | 0x00000000     | 0x00000000     | Hardwired zero — write at step 27 discarded |
| x1       | 0x00000001     | 0x00000001     | addi x1, x0, 1                              |
| x2       | 0x00000005     | 0x00000005     | addi x2, x0, 5                              |
| x3       | 0x00000006     | 0x00000006     | add x3, x1, x2                              |
| x4       | 0x0000000C     | 0x0000000C     | add x4, x2, x6 (step 20)                    |
| x5       | 0x00000003     | 0x00000003     | addi x5, x4, -2                             |
| x6       | 0x00000007     | 0x00000007     | or x6, x5, x4                               |
| x7       | 0x00000001     | 0x00000001     | and x7, x4, x5                              |
| x8       | 0x00000000     | 0x00000000     | slt x8, x4, x5 — false; not in non-zero set |
| x9       | 0x00000001     | 0x00000001     | slt x9, x5, x4 — true                       |
| x10      | 0x00000008     | 0x00000008     | addi x10, x0, 8                             |
| x11      | 0x00000004     | 0x00000004     | sub x11, x2, x7                             |
| x12      | 0x00000006     | 0x00000006     | lw x12, -4(x10) — loads dmem[1] = 6         |
| x13      | 0x00000000     | 0x00000000     | Dead code — andi x13 skipped                |
| x14      | 0x00000005     | 0x00000005     | andi x14, x2, 15                            |
| x15      | 0x0000000F     | 0x0000000F     | ori x15, x4, 3                              |
| x16      | 0xFFFFFFFFFC   | 0xFFFFFFFFFC   | sub x16, x7, x2                             |
| x17–x31  | 0x00000000     | 0x00000000     | Not written                                 |

### 5.4 Final Data Memory (non-zero words)

| Word index | Byte address | Expected (hex) | Observed (hex) | Source instruction |
|------------|--------------|----------------|----------------|--------------------|
| 1          | 0x04         | 0x00000006     | 0x00000006     | sw x3, 4(x0)       |
| 3          | 0x0C         | 0x00000007     | 0x00000007     | sw x6, 4(x10)      |
| 4          | 0x10         | 0x00000008     | 0x00000008     | sw x10, 16(x0)     |
| 5          | 0x14         | 0x00000001     | 0x00000001     | sw x9, 20(x0)      |

## 6. Functional Coverage

### 6.1 Coverage Matrix

| Coverage Point               | Status      | Notes                                      |
|------------------------------|-------------|--------------------------------------------|
| R-type: add                  | COVERED     | Steps 3, 20                                |
| R-type: sub                  | COVERED     | Steps 4, 13, 28                            |
| R-type: and                  | COVERED     | Step 7                                     |
| R-type: or                   | COVERED     | Step 6                                     |
| R-type: slt (false result)   | COVERED     | Step 8                                     |
| R-type: slt (true result)    | COVERED     | Step 9                                     |
| I-type: addi (positive imm)  | COVERED     | Steps 1, 2, 11                             |
| I-type: addi (negative imm)  | COVERED     | Step 5 (-2)                                |
| I-type: addi to x0 (discard) | COVERED     | Step 27: addi x0, x5, 9 — write suppressed |
| I-type: andi                 | COVERED     | Step 17 (andi x14)                         |
| I-type: ori                  | COVERED     | Step 23                                    |
| I-type: slti                 | NOT COVERED | Decodable but not present in program       |
| Load: lw (negative offset)   | COVERED     | Step 14: lw x12, -4(x10) → loads dmem[1]   |
| Store: sw (positive offset)  | COVERED     | Steps 10, 12, 16, 26                       |
| Branch: beq taken            | COVERED     | Step 15: beq x2, x4 (both = 5)             |
| Branch: beq not taken        | COVERED     | Step 24                                    |
| Branch: bne taken            | COVERED     | Steps 19, 22                               |
| Branch: bne not taken        | COVERED     | Step 18: bne x2, x4 (both = 5)             |
| Branch: blt taken            | COVERED     | Step 21: blt x0, x1 (0 < 1)                |
| Branch: blt not taken        | COVERED     | Step 25: blt x2, x1 (5 < 1 false)          |
| x0 write suppression         | COVERED     | Step 27 confirms hardwired-zero behaviour  |
| Registers x17–x31 written    | NOT COVERED | Only x0–x16 are touched                    |
| Data memory word index > 5   | NOT COVERED | Only indices 1, 3, 4, 5 written            |

### 6.2 Dead-Code Lines

Three instructions are never executed: `add x5, x0, x3` at PC 0x3C, `andi x13, x2, 15` at PC 0x40, and `sub x11, x2, x7` at PC 0x54. The first two are skipped by the taken branch at step 15; the third is skipped by the taken branch at step 19. All three are present in the program image and decoded by the reference disassembler but contribute no architectural effect.

## 7. Issues and Recommendations

### 7.1 Open Issues

| # | Issue                          | Details                                                                                                                                                                                                         |
|---|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | slti not exercised             | The I-type ALU slice for slti (funct3=010) is present in both the RTL and the golden model but is never triggered by the current program. A one-instruction extension to the test program would close this gap. |
| 2 | Upper registers unexercised    | Registers x17 through x31 are never written. A follow-on test that exercises the full register file would increase confidence in the register-file address decoding.                                            |
| 3 | Data memory sparsely exercised | Only word indices 1, 3, 4, and 5 are written. A supplementary program that exercises boundary indices (0, 255) and adjacent words would improve confidence in the address decoder.                              |

### 7.2 Recommendations

- Supplement program.mem with a short follow-on sequence that covers slti and at least one upper register (x17+).
- Consider adding a ``timescale` directive to each RTL module to eliminate the Icarus Verilog timing-precision warning that appears when mixed-timescale designs are compiled together.
- Add a fault-injection run: force one register to a wrong value mid-simulation and confirm that the lockstep check flags it immediately.
- Extend data-memory testing to include word index 0 and the upper boundary (index 255) to stress the address decoder.

## 8. Conclusion

The single-cycle RISC-V CPU (`sc_cpu_top_level`) passes all verification checks on the 30-instruction test program. The dual-oracle testbench reported zero errors from the lockstep comparison and zero errors from the independent hand-derived final-state check. The design correctly implements addition, subtraction, bitwise AND/OR, set-less-than, immediate arithmetic, load-word (negative offset), store-word, and all six branch cases (`beq/bne/blt`, both taken and not-taken). The x0 write-suppression path was also exercised. The remaining coverage gaps are minor and are addressed by the recommendations in Section 7.

## Appendix A. Simulation Waveform

The waveform below shows the DUT signal activity captured during the 30-instruction simulation run. Signals are grouped by function: clock, reset, and pc (yellow), instruction decode (violet), control (green), data words (red), and ALU control signals (orange).

